



ಜಿ. ಎಂ. ವಿಶ್ವವಿದ್ಯಾಲಯ
GM UNIVERSITY

P. B. Road, Davanagere – 577 006 KARNATAKA | INDIA

Faculty of Engineering and Technology

School of Computer Science and Technology

Department of Cyber Security and Information Security

Assignment Report – 2

Subject Faculty	Mr. PAVAN KUMAR N T	Semester/Section	4th/
Course	Automata Theory and Computation	Course Code	UE24CS2405
CO Mapped	CO6	Max Marks	10
Issue Date	23/05/2026	Submission Date	10/06/2026



Title: Web-Based Visualizer for Structural Representation of C Programs

Submitted By

Name of the Student: UJJWAL KUMAR

USN : U24E01IY076

COs	CO6	
Allotted Marks for COs	10	
Marks obtained for COs		
Submitted Date	Max Marks	Marks obtained
10/06/2026	10	

Academic Year: 2025-26 (Even Semester)

Ujjwal Kumar
Signature of the Subject Faculty

Signature of the Student

Introduction

Programming languages are designed for human understanding, but computers must process code through several stages before execution. During compilation, source code is converted into internal representations that reveal syntax, control flow and data dependencies. This project presents a web-based visualizer for Transformation, Abstract Syntax Trees (AST), control flow graphs (CFG), Data flow graphs (DFG), and code property graphs (CPG) which are essential in compiler design, program analysis, software engineering, and modern AI-based coding tools. The application allows users to enter a program and observe how code is analyzed internally, helping students better understand compiler construction, automata theory, program structure, and behavior. It also highlights the link between traditional compilers techniques and modern AI code compilers, making it a valuable educational tool. It provides immediate feedback on how different C syntax constructs produce distinct AST nodes, CFG branches, and DFG data dependencies. By studying these visual representations, students develop deeper intuition about how compilers parse, analyze, and optimize code.

Index

- * Introduction
- * objectives
- * Problem statement
- * Lexical Analysis
- * Abstract Syntax Tree
- * Data Flow Graph
- * Code Property Graph
- * Connection to Automata Theory
- * Expected Outcomes
- * Screenshots
- * Conclusion
- * Reference



Objectives

1. To design a web-based application for visualizing the structural representation of C programs
2. To implement lexical analysis, Abstract Syntax Tree (AST), control flow graph (CFG), and Data flow graph (DFG) generation for C source code analysis.
3. To develop a simplified code property graph (CPG) and demonstrate the connection between compiler design, automata theory, and modern AI code assistant

Problem Statement

Understanding how a C program is processed inside a compiler is difficult due to complex stages like lexical analysis, parsing, control flow, and data flow tracking. This project solves that by developing a web-based visualizer that interactively displays tokenization, AST, CFG, DFG and CPG directly in the browser.

Lexical Analysis

Definition

Lexical analysis is the first phase of compilation. The program that performs this work is called a lexer or scanner. It reads the source code character and groups characters into meaningful units called tokens.

A token is the smallest meaningful unit in a programming language.

Working Principle

The lexer uses a concept from automata theory called a Deterministic Finite Automaton (DFA). A DFA is a state machine that:

- Starts in an initial state
- Reads one character at a time.
- Moves to a new state depending on the character
- Accepts a token when it reaches a final state.

Abstract Syntax Tree

Definition

An Abstract Syntax Tree (AST) is a tree-shaped data structure that represents the hierarchical syntactic structure of source code. Unlike a parse tree (concrete syntax tree), the AST does not include every punctuation mark - it keeps only the semantically meaningful parts.

Structure

- Root node - represents the entire program.
- Internal nodes - represent constructs like function, if-statements, and loops.
- Leaf nodes - represent terminals like variable names and literal values.

How it is Built.

The parser (second phase of compilation) reads the token stream from the lexer and applies grammar rules to build the AST. This process is called parsing or syntactic analysis. For C programs, the grammar is a context-free grammar (CFG) - again from automata theory. CFGs are more powerful than regular grammars and can describe nested structures like balanced parentheses.

Control Flow Graph

Definition

A control flow graph (CFG) is a directed graph that models all possible execution paths through a program. It answers the question: "Given this code, what are all the ways it could run?"

Components

Node - A sequence of statements with no branches inside.

Edge - A possible transfer of control between blocks.

Entry node - where execution begins.

Exit node - where execution ends.

Decision node - A point where the path splits.

Back edge - An edge going backward - indicates loop.

Uses of CFG

- Dead code detection - nodes unreachable from start
- Loop detection - back edges indicate loops
- Test coverage - ensuring all execution paths are tested.
- Security analysis - finding "exploitable execution paths."

Data Flow Graph

Definition

A Data Flow Graph tracks how data values flow through a program. Specifically, it tracks:

- DFF - Where a variable is defined
- USE - Where a variable value is used.
- Dependences - which definitions reach which uses.

Uses of DFG

- Compiles optimization - eliminate unnecessary computations.
- Parallelism detection - find statements that can run simultaneously.
- Bug detection - find variables used before definition.
- Security / Taint analysis - tracking untrusted input through the program.

Code Property Graph

Definition

A Code Property Graph (CPG) is a single unified graph that combines the AST (syntax structure), the CFG (control flow) and the DFY (data flow). It was introduced by Fabian Yamaguchi et al. (2014) as a powerful representation for automated code analysis, particularly for finding security vulnerabilities.

Why combine All Three?

Each representation alone is limited:

- AST alone does not tell us execution order
- CFG alone does not tell us variable dependencies.
- DFY alone does not tell us code structure.

Usage in Program Analysis

The CPG is the foundation of tools like:

- Joern - Open-source code analysis platform
- CodeQL (GitHub) - security vulnerability finding at scale.
- Semgrep - pattern-based code scanning.
- Modern AI code assistants - described in section 8.

Connection to Automata Theory

DFA in Lexical Analysis

A Deterministic Finite Automaton (DFA) is formally defined as a 5-tuple: $(Q, \Sigma, \delta, q_0, F)$ where

- Q - Finite set of states
- Σ - Alphabet (set of input characters)
- δ - Transition function: $Q \times \Sigma \rightarrow Q$
- q_0 - Initial / start state
- F - Set of accepting (final) states.

Every token type is defined by a regular expression, and every regular expression has an equivalent DFA.

State Transitions in code Analysis

The CFL is itself a finite automaton where each basic block is a state, edges are transitions, and the program's execution is a path through this automaton.

Data flow analysis uses fixed-point iteration - a concept from lattice theory - to propagate information through the CFG until no more changes occur.

Connection To AI Code Assistants

Tokenization in AI Models

All large language models tokenize code before processing it. However, they use subword tokenization rather than language-specific tokenization. The token `print` might be a single token, or `split` as `print` + `f` depending on frequency in training data. Understanding token boundaries helps the model understand code structure.

How CFT Improves Code Generation

Control flow understanding helps AI assistants:

- Generate correct loop termination conditions
- Avoid generating unreachable code
- Suggest appropriate early returns
- Understand which variables are in scope at each point

How DFL Enables Smarter Suggestions

- Detect when a variable is used before assignment
- Suggest the correct variable name when multiple are in scope
- Understand that changing `x` on one line affects its use on another
- Perform semantic code search (find all uses of variable)

Expected Outputs

Program 1 - Simple Arithmetic.

tokens: ~ 20 tokens including keywords int, identifiers
operators =/+, literals 5/10, symbols;

AST: Program $\rightarrow$ function $\rightarrow$ [Declaration: a,

Declaration: b, Declaration: c, return].

CFG: START $\rightarrow$ fn: main $\rightarrow$ Declare: a $\rightarrow$ Declare: b
Declare: c $\rightarrow$ return 0 $\rightarrow$ END.

DFG: DEF(a) $\rightarrow$ DEF(c); DEF(b) $\rightarrow$ DEF(c).

Program 2 - if/else

CFG: Fork a + if (x > 5) into then/else branches,
merging after.

DFG: DEF(x) flowing into the condition and
both branches.

Program 3 - For loop.

CFG: Block edge from loop update node back
to condition node.

DFG: DEF(i) flowing into condition, loop body,
and print call.

Screenshot

Description

- Screenshot 1 - Home page / Input Panel.
 - ↳ Shows the full application on load.
- Screenshot 2 - Token Table. (Program 1)
 - ↳ Shows the Tokens tab after analysing Program
- Screenshot 3 - AST Tree. (Program 2).
 - ↳ Shows the AST tab for Program 2.
- Screenshot 4 - CFG (Program 2)
 - ↳ Shows the CFG. SVL visualisation
- Screenshot 5 - DFG (Program 1).
 - ↳ Shows the DFG SVL visualisation
- Screenshot 6 - CFG (Program 3)
 - ↳ Shows the CFG tab for Program 3.

CODE:

The image shows two pages of a document that is almost entirely illegible due to extreme fading and noise. The text is arranged in columns, suggesting a list or index. The top page has a header that is partially visible, and the bottom page has a similar header. The text is too faint to transcribe accurately, but it appears to be a structured list of items.

[illegible]

OUTPUT :

< CVIZ >

Web-Based Visualizer for Structural Representations of C Programs

[Lexical Analysis](#) - [Abstract Syntax Tree](#) - [Control Flow](#) - [Data Flow](#) - [Code Property Graph](#)

☒ [Lexical](#)
☒ [AST](#)
☒ [CFG](#)
☒ [DFG](#)
☒ [CPG](#)

Select analyses to run:

☒ Lexical (Tokens)
☒ Syntax Tree (AST)
☒ Control Flow (CFG)
☒ Data Flow (DFG)
☒ Property Graph (CPG)

< CVIZ >

Web-Based Visualizer for Structural Representations of C Programs

[Lexical Analysis](#) - [Abstract Syntax Tree](#) - [Control Flow](#) - [Data Flow](#) - [Code Property Graph](#)

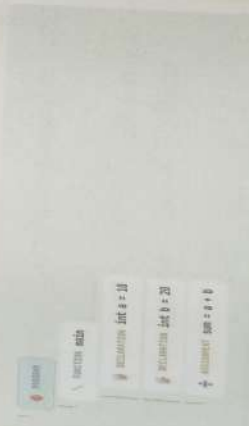
☒ [Lexical](#)
☒ [AST](#)
☒ [CFG](#)
☒ [DFG](#)
☒ [CPG](#)

Select analyses to run:

☒ Lexical (Tokens)
☒ Syntax Tree (AST)
☒ Control Flow (CFG)
☒ Data Flow (DFG)
☒ Property Graph (CPG)

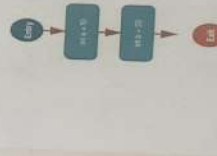
Abstract Syntax Tree

The AST represents the hierarchical syntactic structure of the program, abstracting away punctuation into a tree of meaningful nodes.



Control Flow Graph

The CFG models all possible execution paths. Each node is a basic block, edges represent transfers of control.



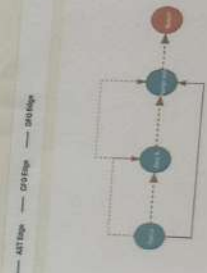
Code Property Graph (CPG)

The CPG represents the semantic structure of the program, abstracting away punctuation into a tree of meaningful nodes.



Code Property Graph (CPG)

The CPG represents the semantic structure of the program, abstracting away punctuation into a tree of meaningful nodes.



Conclusion

This Project demonstrates how a C program is processed internally by a compiler through multiple structured phases. By implementing lexical analysis, AST generation, CFG construction, DFG tracking, and CFG integration in a web-based environment, the system provides a clear and visual understanding of compiler operations.

References

1. Sipser - Introduction to the theory of computation
2. Aho, Lam, Sethi, Ullman - Compilers: Principles, Techniques, and Tools.
3. Yamaguchi et al. - Code property graphs.
4. GitHub codeQL Documentation.
5. Parsing Techniques: A Practical Guide.